

KubSTU CTF Writeup

costavinc

01/05/2026

Contents

1	[Crypto] – Base	3
1.1	Challenge Description	3
1.2	Enumeration & Static Analysis	3
1.3	Exploitation / Solution Path	3
1.4	Solution	3
2	[Crypto] – Strange sequence of numbers	3
2.1	Challenge Description	3
2.2	Enumeration & Static Analysis	3
2.3	Exploitation / Solution Path	3
2.4	Solution	4
3	[Crypto] – Unlucky 13	4
3.1	Challenge Description	4
3.2	Enumeration & Static Analysis	4
3.3	Exploitation / Solution Path	4
3.4	Solution	4
4	[Crypto] – Nintendo 3DS	6
4.1	Challenge Description	6
4.2	Enumeration & Static Analysis	6
4.3	Exploitation / Solution Path	6
4.4	Solution	6
5	[Crypto] – Furry Cipher	7
5.1	Challenge Description	7
5.2	Enumeration & Static Analysis	7
5.3	Exploitation / Solution Path	7
5.4	Solution	8
6	[Crypto] – Not enough part 1	9
6.1	Challenge Description	9
6.2	Enumeration & Static Analysis	9
6.3	Exploitation / Solution Path	10
6.4	Solution	10

7	[Crypto] – Not enough part 2	11
7.1	Challenge Description	11
7.2	Enumeration & Static Analysis	11
7.3	Exploitation / Solution Path	11
7.4	Solution	12
8	[Web] – The Battle of the Strongest	13
8.1	Challenge Description	13
8.2	Enumeration & Static Analysis	13
8.3	Exploitation / Solution Path	13
8.4	Solution	13
9	[Forensics] – Demo	14
9.1	Challenge Description	14
9.2	Enumeration & Static Analysis	14
9.3	Exploitation / Solution Path	14
9.4	Solution	15
10	[Forensics] – Mutant	15
10.1	Challenge Description	15
10.2	Enumeration & Static Analysis	15
10.3	Exploitation / Solution Path	15
10.4	Solution	15
11	[Forensics] – Is the report in English?	16
11.1	Challenge Description	16
11.2	Enumeration & Static Analysis	16
11.3	Exploitation / Solution Path	16
11.4	Solution	16
12	[Forensics] – Tunnel	17
12.1	Challenge Description	17
12.2	Enumeration & Static Analysis	17
12.3	Exploitation / Solution Path	17
12.4	Solution	17
13	[Forensics] – Ratte	18
13.1	Challenge Description	18
13.2	Enumeration & Static Analysis	18
13.3	Exploitation / Solution Path	18
13.4	Solution	18
14	[Forensics] – Vanilla raw	19
14.1	Challenge Description	19
14.2	Enumeration & Static Analysis	19
14.3	Exploitation / Solution Path	20
14.4	Solution	20
15	[Forensics] – Wirepass	20
15.1	Challenge Description	20
15.2	Enumeration & Static Analysis	21
15.3	Exploitation / Solution Path	21
15.4	Solution	21

1 [Crypto] – Base

1.1 Challenge Description

We intercepted a strange message. It seems to be encoded using a popular method. Help us figure out what it says.

1.2 Enumeration & Static Analysis

As we are presented with a txt file, we can easily analyze its contents:

```
1 file ./Base.txt
2 cat ./Base.txt
```

Listing 1: initial file checking

The output clearly shows a string encoded in base64:

```
1 S3ViU1RVKGI0czNfNjRfMXNfdGhlX2JhNWk1KQ==
```

Listing 2: cat output

1.3 Exploitation / Solution Path

Since we are presented with a base64 encoded string, we can easily decode it in Bash.

1.4 Solution

```
1 echo "S3ViU1RVKGI0czNfNjRfMXNfdGhlX2JhNWk1KQ==" | base64 -d
```

Listing 3: cmd command

2 [Crypto] – Strange sequence of numbers

2.1 Challenge Description

I received a strange document with numbers inside — what could it mean?

2.2 Enumeration & Static Analysis

We are presented with one txt file that we can simply analyze its contents:

```
1 file strange_sequence_of_numbers.txt
2 cat strange_sequence_of_numbers.txt
```

Listing 4: initial file checking

The output clearly shows us a list of decimal values:

- **Observation:** These values don't exceed the value 127, and are higher than 33, which fall into the set of printable ASCII characters.

2.3 Exploitation / Solution Path

Since the list of numbers doesn't exceed 127, and the minimum value is 48, we can convert each character from its decimal representation.

2.4 Solution

The flag is simply retrieved:

```
1 flag = "75 117 98 83 84 85 40 97 115 99 49 49 95 99 48 100 51 115 95 97 114 51  
    95 97 110 95 49 110 116 101 114 101 115 116 105 110 103 95 119 52 121 95 116  
    111 95 103 101 55 95 105 110 116 111 95 99 114 121 112 55 48 103 114 97 112  
    104 121 41"  
2  
3 print("".join([chr(int(j)) for j in flag.split(" ")]))
```

Listing 5: exp.py

3 [Crypto] – Unlucky 13

3.1 Challenge Description

13 is an unlucky number. Three layers of encryption, thirteen reasons not to try decrypting this. Something leaked — hopefully it will help you.

3.2 Enumeration & Static Analysis

We are presented with two files, output.txt and encrypt.py. We can analyze their contents:

```
1 file output.txt  
2 cat output.txt  
3 cat encrypt.py
```

Listing 6: initial file checking

The following observations can be drawn:

- **Observation 1:** From the cat of the file output.txt, we understand that it might be used an algorithm similar to the RSA algorithm, since we are presented with the modulus n , exponent e , and ciphertext c ;
- **Observation 2:** From the cat of the encrypt.py we can observe the sequence of encrypting actions that have been taken to encrypt the flag.

3.3 Exploitation / Solution Path

From the encrypt.py, we can observe that the number 13 is used as seed for the pseudo random number generator (prng). In the first layer, the values generated by the prng are xorred with the flag characters; in the second layer, the customized forgotten cipher is applied to the data produced from the first layer. Finally, the value produced from the second layer is converted to a decimal value, and transmitted in RSA fashion, that is $c = m^e \bmod N$. Since the ciphertext produced is much smaller with respect to the modulus N , we can obtain the value of m by taking the cubic root of c . The forgotten cipher creates a mesh of Substitution boxes driven by the secret key, which is known, and thus fully revertible. Finally, the first layer is also revertible since the seed is known, and we can generate the same sequence of prng sequence.

3.4 Solution

```
1 import hashlib, math, gmpy2  
2 from decimal import Decimal  
3 from Crypto.Util.number import long_to_bytes, bytes_to_long  
4
```

```

5 n = 136586330371317880323516184270722474767179545423964086335607738843645545590
  7051140133813106890951971486343903298084252286158050613588858876018079105778
  7733146681063635707678102393673406833618170820088193214167654538266905388169
  6220813144460118334045062114722579993438053542378841720115407895675310499722
  85279517155764888481047450059
6 e = 3
7 c = 581064029452524128858679080421167948194643057449718995780730203040675435480
  7080745717865856348815704073130926760080487520249019185196462907134890718334
  8604959890636799938893288492152226256276862912678404321252232876509325092153
  1648136353604710854983368532200782066204580278766014426983094834523132019633
  51276803179820555434275959791505894082437152805771101360567738286600728
8
9 FLAG=b"0000000000000000"
10
11 def cursed_prng(seed, length):
12     state = seed
13     stream = []
14     for _ in range(length):
15         state = (state * 1313 + 131313) % (2**32)
16         stream.append(state & 0xFF)
17     return bytes(stream)
18
19 UNLUCKY_NUMBER = 13
20
21 #layer1 = bytes(a ^ b for a, b in zip(
22 #     FLAG,
23 #     cursed_prng(UNLUCKY_NUMBER, 64)#len(FLAG))
24 #))
25
26 def get_flag(layer1):
27     res = bytes(a ^ b for a,b in zip(layer1, cursed_prng(UNLUCKY_NUMBER,64)))
28     return res
29
30 def forgotten_cipher(key, data):
31     S = list(range(256))
32     j = 0
33     for i in range(256):
34         j = (j + S[i] + key[i % len(key)]) % 256
35         S[i], S[j] = S[j], S[i]
36     i = j = 0
37     out = []
38     for byte in data:
39         i = (i + 1) % 256
40         j = (j + S[i]) % 256
41         S[i], S[j] = S[j], S[i]
42         out.append(byte ^ S[(S[i] + S[j]) % 256])
43     return bytes(out)
44
45 secret = b"Unlucky" + str(UNLUCKY_NUMBER).encode()
46 fc_key = hashlib.sha256(secret).digest()[:16]
47
48 m, exact = gmpy2.iroot(c, 3)
49 if exact:
50     print("Message found:", m)
51
52 int_m = int(m)
53 layer1 = forgotten_cipher(fc_key, long_to_bytes(int_m))
54 FLAG = get_flag(layer1)
55 print(FLAG)

```

Listing 7: exp.py

4 [Crypto] – Nintendo 3DS

4.1 Challenge Description

Hey, I can never remember the name of the algorithm, but it's something very similar to Nintendo 3DS — help me figure out what was written there ^^.

4.2 Enumeration & Static Analysis

As we are presented with a txt file, we can easily analyze its contents:

```
1 file ./3ds_output_2.txt
2 cat ./3ds_output_2.txt
```

Listing 8: initial file checking

The output clearly shows us a list of useful information:

- **Observation 1:** It was used the CBC cipher mode, with a PKCS5 padding scheme;
- **Observation 2:** We are presented with three values (1,2,3), which probably represent the hidden key, encoded in a different manner - string 1 is encoded in base64, characters of value 2 are encoded in decimal form, and string 3 is encoded in hexadecimal;
- **Observation 3:** We are presented with two values for the initialization vectors, ivx and ivm, of which ivx is encoded in hexadecimal and ivm is already in plaintext.

4.3 Exploitation / Solution Path

As the description hints, the cryptographic algorithm involves a triple DES cipher. A 3DES cipher is a 64-bit symmetric block cipher that applies three times the same encryption/decryption algorithm - its key size is 168 bits long (56 x 3), and nowadays its usage is deprecated with respect to the most secure AES algorithm. First we start to decode what is decodable.

```
1 echo "TjFudDNuZG8==" | base64 -d
```

Listing 9: cmd command

```
1 "".join(chr(x) for x in (83, 51, 99, 117, 114, 49, 116, 121))
2 print(bytes.fromhex("4b33792132303236").decode('utf-8'))
3 key = var_1 + var_2 + var_3
```

Listing 10: python commands

The master key is then obtained from the concatenation of the three values obtained after decoding. The initialization vector is also decoded, and since it is used the CBC mode, it means that the initialization vector must be at most 64 bits long: this requires us to XOR the two iv strings:

```
1 ivx = bytes.fromhex("0a001f0273760054")
2 iv = bytes([a ^ b for a,b in zip(ivx,ivm)])
```

Listing 11: python command

4.4 Solution

```
1 from Crypto.Cipher import DES3,DES
2 from Crypto.Util.Padding import pad, unpad
3 from Crypto.Random import get_random_bytes
4
5 var_1 = b"Nint3ndo"
```

```

6 var_2 = b"S3curity"
7 var_3 = b"K3y!2026"
8
9 key = var_1 + var_2 + var_3
10
11 ivx = bytes.fromhex("0a001f0273760054")
12 ivm = b"M4r10Br0"
13 iv = bytes([a ^ b for a,b in zip(ivx,ivm)])
14 #print(iv) #b'G4m3C4rd'
15
16 cipher = bytes.fromhex("072a8e75459a545679f3aa56a9fafb38871022de0c9bd5d7ef55e8d
    ad7861662eb0fb630d9cdf9dd8c64a3a8ac28b86a")
17
18 cipher_decrypt = DES3.new(key, DES3.MODE_CBC, iv)
19 decrypted_padded = cipher_decrypt.decrypt(cipher)
20 original_data = unpad(decrypted_padded, DES3.block_size, style='pkcs7')
21
22 print(f"Original Message: {original_data.decode('utf-8')}")

```

Listing 12: exp.py

5 [Crypto] – Furry Cipher

5.1 Challenge Description

I was scrolling through my email and saw a message from FurryHater_2009) asking to decrypt some strange message, along with two attached files.

5.2 Enumeration & Static Analysis

We are presented with two files - a Furry_Cipher.py, and Weird_Furry_text.txt

```

1 file ./Furry\Cipher.py
2 cat ./Furry_Cipher.py
3 file ./Weird_Furry_text.txt
4 cat ./Weird_Furry_text.txt
5 strings ./Weird_Furry_text.txt
6 ls -la

```

Listing 13: initial file checking

The output clearly shows us a list of useful information:

- **Observation 1:** The output of cat on Furry_Cipher.py shows us that a personalized algorithm was used for the encryption of the flag string;
- **Observation 2:** The output of cat on Weird_Furry_text.txt shows us that the file is padded with garbage, and there is no point in visually analysing what is there;
- **Observation 3:** Also, the output of strings doesn't help us, as the padding characters are all printable characters.

5.3 Exploitation / Solution Path

The exploitation is divided in two parts. In the first part, the target string must be retrieved from all the garbage that is present in Weird_Furry_text.txt; the second part reverts back the encryption algorithm (and thus recovers the hidden flag).

As we inspect the cipher, we notice that it maps printable characters that fall in the interval [a-zA-Z0-9], to other characters that are in the same interval. Since the characters [()_] remain unchanged, we can add them to the string search [a-zA-Z0-9()_].

```

1 num = char_map[char]
2 key_val = key_values[i % 3]
3 if i % 3 == 0:
4     encrypted = (num * 13 + key_val * 7) % 62
5 elif i % 3 == 1:
6     encrypted = (num * 17 + key_val * 3 + 11) % 62
7 else:
8     encrypted = (num * 19 + (key_val ^ 42) + 23) % 62

```

Listing 14: personalized cipher - key selection and addition

For what concerns the encryption algorithm, it can be noticed that the key is a vector of length 3, and its key values fall in the range $[0,61]$ (due to the modulus application). The combination of the key values can be searched with brute force, as it would require $62*62*62 = 186$ attempts in total. In order to recover the character mappings, we need to leverage our knowledge about the computation of modular multiplicative inverse. That's possible due to the fact that the greatest common divisor (gcd) between (13 and 62, 17 and 62, and 19 and 62) is equal to 1, which means they are coprime. By computing the extended gcd (egcd) between them, for e.g. between 13 and 62, we obtain:

$$\text{egcd}(13, 62) = 1 = a * 62 + b * 13 = 4 * 62 + (-19) * 13$$

we can find a value "b" such that $b * (\text{encrypted} - \text{key_val} * 7) \% 62 = \text{num}$. In this case, -19 is the inverse modulo 62 of 13, as $-19 * 13 \bmod 62$ is 1. Since the part related to the key gets cancelled under the hypothesis of correct key values, as a result we will obtain the correct mapping *num*, which can be correctly inverted.

5.4 Solution

We can search our hidden flag as follows:

```

1 import re
2
3 pattern = re.compile(r"[a-zA-Z0-9()_]" )
4 res=""
5 with open("Weird_Furry_text.txt", "r") as file:
6     for line in file:
7         for match in pattern.finditer(line):
8             res += match.group()
9 print(res)

```

Listing 15: string recovery

And follows the decryption of the flag:

```

1 import string
2
3 def encrypt_custom(plaintext, key_values):
4     alphabet = string.ascii_uppercase + string.ascii_lowercase + string.digits
5     char_map = {ch: i for i, ch in enumerate(alphabet)} #A:0
6     num_map = {i: ch for i, ch in enumerate(alphabet)} #0:A
7     result = []
8     for i, char in enumerate(plaintext):
9         if char in '()_':
10             result.append(char)
11         elif char in char_map:
12             num = char_map[char]
13             key_val = key_values[i % 3]
14             if i % 3 == 0:
15                 encrypted = (num * 13 + key_val * 7) % 62
16             elif i % 3 == 1:
17                 encrypted = (num * 17 + key_val * 3 + 11) % 62

```



```

18         else:
19             encrypted = (num * 19 + (key_val ^ 42) + 23) % 62
20             result.append(num_map[encrypted])
21         else:
22             result.append(char)
23     return ''.join(result)
24
25 def decrypt(ciphertext, key_values):
26     res = []
27     index = 0
28     alphabet = string.ascii_uppercase + string.ascii_lowercase + string.digits
29     char_map = {ch: i for i, ch in enumerate(alphabet)} #A:0
30     num_map = {i: ch for i, ch in enumerate(alphabet)} #0:A
31     for i, char in enumerate(ciphertext):
32         if char in '()_':
33             res.append(char)
34         elif char in char_map:
35             encrypted = char_map[char]
36
37             if i%3 == 0:
38                 num = (encrypted*(-19) - -19*7*key) % 62
39             elif i%3 == 1:
40                 num = (encrypted*11 - 11*(11 + 3*key)) % 62
41             else:
42                 num = (encrypted*(-13) - (-13)*(23 + 42^key)) % 62
43
44             res.append(num_map[num])
45         else:
46             pass
47         index += 1
48     return "".join(res)
49
50 if __name__ == "__main__":
51     flag = "XiEDJ5(9tV_qY3_v43_t9B3_o9vo_ESM_YR_YA_t_S5t8v_XYL4jt)"
52     example_text = "Test123"
53     example_key = [1, 2, 3] #they wrap on 62
54     encrypted_example = encrypt_custom(example_text, example_key)
55     print(encrypted_example)
56
57     for i in range(63):
58         for j in range(63):
59             for k in range(63):
60                 computed = decrypt(flag, [i, j, k])
61                 if "KubSTU" in computed:
62                     print(decrypt(flag, [i, j, k]))

```

Listing 16: exp.py

6 [Crypto] – Not enough part 1

6.1 Challenge Description

During a system dump, some parameters were damaged: only part of the bits of p were preserved — the last 72 bits were lost. Then, from the recovered parameters, a key was derived by hashing the secret (this also survived — I think it's AES-GCM?)

6.2 Enumeration & Static Analysis

We are presented with one txt file:

```
1 file output.txt
```

```
2 cat output.txt
```

Listing 17: initial file checking

From the output file, we draw the following conclusions:

- **Observation 1:** target server used the parameters for RSA algorithm (N , e , p_{hi});
- **Observation 2:** we are presented with the parameters used for the encryption of the flag through AES-GCM algorithm.

6.3 Exploitation / Solution Path

Only the higher part of the prime is known, but this doesn't stop an attacker from the retrieval of the entire value of p if it is known at least half sizes of the original p . Since the size of roughly 512 bits, the last 72 bits can be easily retrieved through the Coppersmith attack. The retrieval of the missing part of p is performed in `sagemath`, while the retrieval of the flag in `python`.

6.4 Solution

The missing part of p is retrieved by setting up a polynomial ring shifted left by 72, and added the incognito x .

```
1 N = 151708532784988710186354895816447243710932251919277531742510058529452761722
  4324395264542513123770079659429295124945812887445048811597692968736534695214
  4012243293107344334010132437439958271445412894985600404452597154167300754281
  2412410730741409339131837689130677766341334202879471841791194383321275827852
  931391
2
3 p_hi = 261585037973132772577820331336578451283892270218501125702993194784612273
  6769634242897120915009678837272252972592332233245419497072070
4
5 p_known = p_hi << 72
6 R.<x> = PolynomialRing(Zmod(N))
7 f = p_known + x
8 roots = f.small_roots(X=2^72, beta=0.3, epsilon=0.01)
9 if roots:
10     x0 = roots[0]
11     p = p_known + Integer(x0)
12     print(f"Recovered p: {p}")
13     print(f"Factor q: {N // p}")
14     print(f"Root value: {x0}")
15 else:
16     print("Could not recover p.")
```

Listing 18: recover.sage

As the prime p is recovered, follows the recovery of all the remaining parameters, and the extraction of the flag:

```
1 from Crypto.Cipher import AES
2 from Crypto.Util.number import long_to_bytes, bytes_to_long
3 import hashlib
4 import binascii
5
6 N = 151708532784988710186354895816447243710932251919277531742510058529452761722
  4324395264542513123770079659429295124945812887445048811597692968736534695214401
  2243293107344334010132437439958271445412894985600404452597154167300754281241241
  0730741409339131837689130677766341334202879471841791194383321275827852931391
7 e = 65537
8
9 p_hi = 261585037973132772577820331336578451283892270218501125702993194784612273
  6769634242897120915009678837272252972592332233245419497072070
```

```

10 p_lo = 1277292877421571572747
11
12 p = 123530041574450559800504879403701186974161501594698575119589473819974628568
3200863839988331123090393499596295274285163456780510907418353779502258303464346
7
13 q = 122811043250200155554663776035960005780917728411167286397876578361074215172
7214784134305587308757402248837372744597753511123674624125008273623043384891497
3
14
15 nonce = bytes.fromhex("8fe6c8d25d0738576b6f6a25")
16 phi = (p-1)*(q-1)
17 d = pow(e,-1, phi)
18 tag = bytes.fromhex("9755d120d8356f29ca31eacff3360ab0")
19 ciphertext = bytes.fromhex("0094dfe5f358aecb96369cf72731d114bf0a0008cbe1d15b98b
30f4fd1492e0ee1567a7fd602dc3ff7aa709ea98e7c06eb261c")
20
21 kdf = hashlib.sha256(long_to_bytes(d)).digest()[0:16]
22
23 try:
24     cipher = AES.new(kdf, AES.MODE_GCM, nonce=nonce)
25     plaintext = cipher.decrypt_and_verify(ciphertext, tag)
26     print("Decryption successful:", plaintext.decode('utf-8'))
27 except ValueError:
28     print("Decryption failed: Key is incorrect or message has been corrupted.")

```

Listing 19: exp.py

7 [Crypto] – Not enough part 2

7.1 Challenge Description

During an emergency system recovery, only fragments of two ***-keys were saved. From each prime number, only the high bits survived, and the lower bits were ??????. After recovering the private parameters, a master secret was formed from them, which was then converted via KDF into a key for something.

7.2 Enumeration & Static Analysis

We are presented with one txt file:

```

1 file output_1.txt
2 cat output_1.txt

```

Listing 20: initial file checking

From the output file, we draw the following conclusions:

- **Observation 1:** target server used the parameters for the RSA algorithm (N, e, p_hi);
- **Observation 2:** we are presented with the parameters used for the encryption of the flag through AES-GCM algorithm.

7.3 Exploitation / Solution Path

Also this time only the higher part of the prime p is known, so in both cases we are able to retrieve the missing parts. For the first prime p, it can be easily used the previous technique for "Not enough part 1". For the second prime p, it is unknown the length of the missing part. Then, in a for cycle, it can be bruteforced different shifting lengths, until a solution is found.

7.4 Solution

```

1 N = 107693247573615587212525584706568693993099179606983949136202089061406798764
  0114064802359844792408506916253675569831679851151659289056041533252485095437
  3194315297276041054371296476595148735249560340665463879293890974783532066203
  3945154261022711277503725296902639416639681217584931348322829694077692764339
  287931
2 p_hi = 832313695256042953182732157279176245412116775736305489730893867560396386
  8132925956974213137032081376425627149514834712272012976728
3
4 for shift_ in range(1,300):
5     p_known = p_hi << shift_
6     R.<x> = PolynomialRing(Zmod(N))
7     f = p_known + x
8     roots = f.small_roots(X=2^shift_, beta=0.2, epsilon=0.05)
9     if roots:
10        x0 = roots[0]
11        p = p_known + Integer(x0)
12        print(f"Recovered p: {p}")
13        print(f"Factor q: {N // p}")
14        print(f"Root x0: {x0}")
15        break
16    else:
17        print(f"Could not recover p. for shift: {shift_}")

```

Listing 21: recover.sage

As the prime p is recovered, follows the recovery of all the remaining parameters, and the extraction of the flag:

```

1 from Crypto.Cipher import AES
2 from Crypto.Util.number import long_to_bytes, bytes_to_long
3 import hashlib
4 import binascii
5
6 N_1 = 8290335680764442751129177376137892202942261118835804722155324212203641838
  6827855049155897729546477938247313902660497920739772848211750220810305389462
  5314169899024653585542690279580640391846535009000063677004150495315221359239
  5391362300366613385018021680016477185552302034873978687967760190691163909768
  9182081
7 e_1 = 65537
8 p_1 = 1168293458782231591137577728363872274290584146165062929579200996496634933
  6971698849336863242750472746239661943288355824449431578999081220550343245425
  377599
9 q_1 = 7096107248093178561280921148906200357200763243065263189174354891958252150
  3309954892006955924919517627433297115945936582943216558507974366725776320334
  72319
10
11 phi_1 = (p_1 - 1)*(q_1 - 1)
12 d_1 = pow(e_1, -1, phi_1)
13
14 N_2 = 1076932475736155872125255847065686939930991796069839491362020890614067987
  6401140648023598447924085069162536755698316798511516592890560415332524850954
  3731943152972760410543712964765951487352495603406654638792938909747835320662
  0339451542610227112775037252969026394166396812175849313483228296940776927643
  39287931
15 e_2 = 65537
16 p_2 = 1006205516213892421487482484780719679528695512647502629949612619629855779
  5094160854981099370247925487576197518465890392600093676556658169963127903963
  728563
17 q_2 = 1070290769015451101785493711187477592497747808409464024560351075871464594
  2592948520012503707969619462265381655541832814234527211077932099308308287837
  299737
18

```

```

19 phi_2 = (p_2 - 1)*(q_2 - 1)
20 d_2 = pow(e_2, -1, phi_2)
21
22 key = long_to_bytes(d_1) + long_to_bytes(d_2)
23
24 nonce = bytes.fromhex("492d206feb1c86188589bb46")
25 tag = bytes.fromhex("02d3cd6b6aaa54d7026029a16fba111b")
26 ciphertext = bytes.fromhex("4bbbb06c1b54b2bca2eab8f6cd2e7bb90f269bf7f8a7661ef84
    f78a554bbc6dca350f70e2f0df47add8bca0a131ac821bdf20f97e5e20215")
27
28 kdf = hashlib.sha256(key).digest()[:16]
29
30 try:
31     cipher = AES.new(kdf, AES.MODE_GCM, nonce=nonce)
32     plaintext = cipher.decrypt_and_verify(ciphertext, tag)
33     print("Decryption successful:", plaintext.decode('utf-8'))
34 except ValueError:
35     print("Decryption failed: Key is incorrect or message has been corrupted.")

```

Listing 22: exp.py

8 [Web] – The Battle of the Strongest

8.1 Challenge Description

Every semester, capybara students argue about which faculty is better, who's stronger academically, who's more active in student life.

This year, enthusiast capybaras decided to take the rivalry digital and created an innovative service "The Battle of the Strongest". But it seems like the service isn't the most secure one.

8.2 Enumeration & Static Analysis

Since it's a web challenge, it will be used Burp Suite to draw an initial mapping of requests that are managed by the site. At first sight, we are presented with a 5 minute timer, after which the site promises us to reveal the flag if we guess the total amount of likes at timer's end.

8.3 Exploitation / Solution Path

From Burp we can identify the most useful API endpoints we will need in order to solve the challenge:

- **Likes:** <http://155.XXX.XXX.XXX/api/likes>
- **Timer:** <http://155.XXX.XXX.XXX/api/timer>
- **Bet:** <http://155.XXX.XXX.XXX/api/bet>

Where, in turn, we will use the method GET with /api/likes and /api/timer, and the method POST with /api/bet. Once we are close to the timer's end, we will need to get the exact amount of likes from the server, and send it back as fast as possible.

8.4 Solution

Do not run code unless you fully understand it, and if you are aware which are the possible consequences

```

1 import requests
2 import time
3
4 s = requests.Session()
5 s.cookies['session'] = "eyJ1c2VyX2lkIjoxMDIsInVzZXJuYW11Ijo1YSJ9.afRjHw.UUmdfru
   fo5Egtweb1LdYx5mxSMo"
6 s.cookies['web_session'] = "ab59b291cc3af8ac"
7
8 def get_likes():
9     return s.get('http://155.XXX.XXX.XXX/api/likes')
10
11 def get_time():
12     return s.get('http://155.XXX.XXX.XXX/api/timer')
13
14 def set_val(n):
15     payload = {"bet":n}
16     return s.post("http://155.XXX.XXX.XXX/api/bet", json=payload)
17
18 print(get_likes().status_code)
19 print(get_likes().json())
20 print(get_time().json())
21 get_time().json()
22
23 while (get_time().json()['remaining'] > 5.0):
24     time.sleep(3)
25     print(get_time().json())
26     print(get_likes().json()['count'])
27
28 while (get_time().json()['remaining'] < 5.0):
29     #follow the obvious actions by means of get_likes(), and set_val()

```

Listing 23: exp.py

9 [Forensics] – Demo

9.1 Challenge Description

During a security audit, suspicious actions were discovered on the company’s web server. It is believed that the attacker was able to penetrate the network, move laterally to the database server, and steal confidential information.

Flag format: KubSTU....

Identify what vulnerability was used to gain initial access and what was uploaded. Under which user did the attacker operate next? What was copied? Example: KubSTUXSS,p0wny.php,Administrator.

9.2 Enumeration & Static Analysis

Once we unrar the Demo.rar, we are presented with a list of files.

```

1 unrar ./Demo.rar
2 ls -la

```

Listing 24: initial file checking

Luckily for us, the extension of the files allows their inspection without the usage of any particular tool other than cat.

9.3 Exploitation / Solution Path

So, as hinted in the description, we must recover the attack chain. The first files that might help us understand what has happened are access.log and error.log. Here we can observe which

errors occurred in the system, and triggered by which payloads. After which, it can be easily identified which file has been uploaded. In auth.log it can be identified with which account the attacker gained access to the system. Finally, as an attacker is inside the system, it may be worth looking at .bash_history, as it shows with which files the attacker has interacted with.

9.4 Solution

```
1
2 cat access.log | grep 200
3 cat access.log | grep shell
4 cat access.log | grep sqlmap
5 cat auth.log | sort | grep Accepted
6 cat .bash_history
```

Listing 25: file inspection

10 [Forensics] – Mutant

10.1 Challenge Description

Congratulations! You've been admitted to the polytechnic for information security. You've been given educational material. The exam is tomorrow — good luck.

10.2 Enumeration & Static Analysis

The text inside the file crypt.pdf doesn't say much. We can dig deeper:

```
1 file ./crypt.pdf
2 strings ./crypt.pdf
3 cat ./crypt.pdf
```

Listing 26: initial file checking

From the output we can notice several things:

- **Observation 1:** There is an encoded stream object that does not appear when the file is opened normally;
- **Observation 2:** The stream is enclosed by the characters "<~" and "~>", and it explicitly states that it is an encoded part.

10.3 Exploitation / Solution Path

We must understand which kind of encoding was used, and manually revert it back. Thankfully, google easily shows us the right direction if we query which kind of encoding wraps a stream between "<~" and "~>" characters.

10.4 Solution

As we identify the stream object, follows its decoding:

```
1 import zlib, base64
2
3 def get_flag(x):
4     s = ""
5     for el in x:
6         offset = el.find("(")
7         s += el[offset+1]
8     return s
```

```

9
10 with open("crypt.pdf", "r") as mut:
11     data = mut.read()
12
13 obj5_start = data.find("5 0 obj")
14
15 stream_start = data.find('stream', obj5_start) + len("stream")
16 stream_end = data.find('endstream', stream_start)
17 stream = data[stream_start:stream_end].strip()
18 print(stream[2:-2])
19
20 # For ASCII85 decoding (identified by the [<~,~>] keys)
21 decoded_ascii85 = base64.a85decode(stream[2:-2])
22 original_data = zlib.decompress(decoded_ascii85).decode('utf-8')
23
24 for x in original_data.split("\n"):
25     print(x)
26 print(get_flag(original_data.split("\n")))

```

Listing 27: exp.py

11 [Forensics] – Is the report in English?

11.1 Challenge Description

A strange file arrived by email — a financial report, and in English at that.

11.2 Enumeration & Static Analysis

We are presented with only a pdf file, and nothing else.

```

1 file ./KUBSTU_Financial_Report_2025.pdf
2 strings ./KUBSTU_Financial_Report_2025.pdf
3 cat ./KUBSTU_Financial_Report_2025.pdf

```

Listing 28: initial file checking

From the output we can notice several things:

- **Observation 1:** There are a lot of fake flags;
- **Observation 2:** There is an embedded file hidden in the report.

11.3 Exploitation / Solution Path

As there is a hidden file inside, it is worth scanning the whole pdf in search for file signatures. The embedded file that was identified in the previous step, suggests us that it may be the case of a zip file.

11.4 Solution

Thanks to binwalk, we can identify which file is hidden, and at which offset the zip file does start:

```

1 binwalk KUBSTU_Financial_Report_2025.pdf

```

Listing 29: pdf scanning

It follows the archive extraction:


```
1 dd if=KUBSTU_Financial_Report_2025.pdf of=extracted_archive.zip bs=1 skip=2122
   count=577
```

Listing 30: archive extraction

Then we can unzip the file by using the password that we find in the pdf

```
1 unzip UBSTU_Financial_Report_2025.pdf
```

Listing 31: unzipping

12 [Forensics] – Tunnel

12.1 Challenge Description

Our information security department detected suspicious activity on one of the workstations. It appears the attacker managed to exfiltrate some data using a non-standard communication channel.

12.2 Enumeration & Static Analysis

```
1 strings Krasnodar.pcap
2 wireshark Krasnodar.pcap
```

Listing 32: initial file checking

From the output we can notice several things:

- **Observation 1:** There is literally a lot of noise;
- **Observation 2:** The amount of packets transmitted is huge;

12.3 Exploitation / Solution Path

From the conversation statistics interface, we can notice a huge amount of packets that have been transmitted. If we look through the TCP packets, we notice that the transmitted data is encoded in base64, which if decoded gives us useless values. Then, if we analyze the UDP packets, we notice a lot of connections from port 53. A quick look is enough to observe that data has been exfiltrated from port 53. Each smuggled string has the form base64_chunk.exfiltrate.kubstu-ctf.ru. However, after a more detail inspection, it can be noticed that some smuggled data have form vXX.hex_chunk.exfiltrate.kubstu-ctf.ru, which may be related to our flag. We can recompose the hex string by following the order indicated by vXX.

12.4 Solution

As we gather all the DNS packets, we recompose them following the order v01, v02, etc.

```
1 import io, struct, pyzipper, base64
2 from scapy.all import rdpcap, DNS, UDP
3
4 packets = rdpcap("Krasnodar.pcap")
5 segments = []
6 for i, pkt in enumerate(packets, 1):
7     if pkt.haslayer(DNS) and pkt[UDP].dport == 53:
8         segments.append((i, pkt[DNS].qd.qname))
9
10 segments.sort(key=lambda x: x[0])
11
12 done = set()
```

```

13 extracted = []
14
15 for seq, data in segments:
16     if seq not in done:
17         if data[0] == ord('v') and data[1:3].isdigit() and data[3] == ord('.'):
18             extracted.append((data[1:3], data[4:8]))
19
20 extracted.sort(key=lambda x: x[0])
21 print(extracted)
22
23 flag = b"".join([a[1] for a in extracted])
24 print(flag)
25
26 print(bytes.fromhex(flag))

```

Listing 33: exp.py

13 [Forensics] – Ratte

13.1 Challenge Description

You are an incident response specialist. Your company received a network traffic dump (pcap file) captured from one of the corporate network segments during suspicious activity.

Once again we have suspicious traffic on the network. Everything seems standard, but there's something that doesn't fit.

13.2 Enumeration & Static Analysis

We are presented with a pcap file that we can analyze with Wireshark.

```

1 strings Ratte.pcap
2 wireshark Ratte.pcap

```

Listing 34: initial file checking

From the output we can notice several things:

- **Observation 1:** the output of strings doesn't say much;
- **Observation 2:** From an initial analysis, nothing particular does pop up;

13.3 Exploitation / Solution Path

It is worth to analyze the conversation statistics. We can notice that there is one particular TCP connection that differs from all the others, as it involves around 944 bytes of data transmission. In Wireshark, we can isolate all the packets having as destination port 1337 by inserting the string `tcp.port == 1337` in the filter. We noticed in the packet hexdump that it's present the string `0xdeadbeef`, followed by `0x42`, while the following packets all start with byte `0xcc` and 4 random bytes. `0xdeadbeef` is usually used as a magic number, marking the start of data transmission, while `0x42` may indicate the XOR key that's been applied. In this case, `0xcc` may indicate the start of the stream data.

13.4 Solution

Following all the assumptions that have been taken in the previous step, once we obtain the plaintext, we need to extract the flag by extracting it by a fixed scheme:

```

1 import io
2 import struct
3 import pyzipper
4
5 from scapy.all import rdpcap, TCP, Raw
6
7 packets = rdpcap("Ratte.pcap")
8 segments = []
9 for i,pkt in enumerate(packets,1):
10     if pkt.haslayer(TCP) and pkt[TCP].dport == 1337:
11         segments.append((i, bytes(pkt[TCP].payload)))    #No., tcp_data
12
13 segments.sort(key=lambda x:x[0])
14 print(segments)
15
16 done = set()
17 stream = b""
18 for seq, data in segments:
19     if seq not in done:
20         done.add(seq)
21         stream += data
22
23 print(" ".join(f'{b:02x}' for b in stream))
24
25 magic = stream[:4]
26 xor_key = stream[4:4+1]
27 encrypted = stream[5:]
28 print(magic, xor_key, encrypted, len(encrypted))
29
30 plaintext = bytes([a ^ int.from_bytes(xor_key) for a in encrypted])
31 offset = 0
32 flag = b""
33 for b in plaintext:
34     flag += plaintext[offset+3:offset+3+2]
35     offset+=5
36
37 print(flag)

```

Listing 35: exp.py

14 [Forensics] – Vanilla raw

14.1 Challenge Description

We received a RAM dump, but for some reason we can't analyze it — help us out.

14.2 Enumeration & Static Analysis

We only have a rar archive which, once unrarred, it gives us a memory dump memory.raw. The memory dump is around 2GB big, so its quite cumbersome to handle with the usual tools from cmd.

```

1 file memory.rar
2 unrar memory.rar
3 ls -la

```

Listing 36: initial file checking

14.3 Exploitation / Solution Path

We know the format of the flag, "Kubstu", so we can try to look for a string that matches such pattern. We can start with matching the letter "K", and see what comes up.

14.4 Solution

We need to find the offset value for each match we find

```
1 with open("memory.raw", "rb") as memory:
2     chunk = 1024*1024
3     offset = 0
4     while True:
5         data = memory.read(chunk)
6         if not data:
7             print("nothing")
8             break
9
10        for i, b in enumerate(data):
11            if b != 0 and ord("K") == b:
12                print(offset+i)
13                start_pos = offset + i
14                memory.seek(start_pos) #so as not read evrything
15                print(f"Chunk size: {offset}")
16                payload = memory.read(64)
17                #print("String found: " + " ".join(f'{b:02x}' for b in payload))
18
19                print(f"Plain value: {payload}")
20                #exit()
21            offset += chunk
```

Listing 37: exp.py

If we inspect each offset with xxd, we can find that for one particular offset it is hidden the flag by padding each character with 3 bytes:

```
1 xxd --seek 1039150061 -l 100 memory.raw
```

Listing 38: hex dump at found offset

Once we identify the scheme, the flag can be easily retrieved:

```
1 with open("memory.raw", "rb") as memory:
2     offset = 1039150061
3     flag = ""
4     memory.seek(offset)
5     to_extract = memory.read(200)
6     for i in range(0, 40):
7         flag += chr(to_extract[i*4])
8     print(flag)
```

Listing 39: exp.py

15 [Forensics] – Wirepass

15.1 Challenge Description

Arctic intelligence detected anomalous activity in the secret network of the Penguin Command. According to intelligence sources, two field infrastructure nodes were transferring operational documents related to a military operation against Capybarovsk.

Our analysts managed to intercept a network dump, but it turned out the operatives weren't so simple: data was transmitted over an encrypted channel using a custom protocol.

15.2 Enumeration & Static Analysis

We can analyze the pcap file with the usual tools:

```
1 strings -n15 challenge.pcap
2 wireshark challenge.pcap
```

Listing 40: initial file checking

From the output we can notice several things:

- **Observation 1:** We can notice some plaintext data transmission, but nothing worth our attention;
- **Observation 2:** The amount of packets that have been transmitted is relatively low, but the volume of data is quite big.

15.3 Exploitation / Solution Path

From the conversation statistics interface, we can analyze the transmitted packets. We can notice that the biggest amount of data is transmitted to destination port 31337. This data can be extracted directly from wireshark or python. We can notice that the first 4 bytes represent the word "XFER" as a magic number, so it can be assumed that the following 8, 16, or 32 bytes may work as a key for the following dump.

This means we are still missing the decryption key of the archive. We can reduce the search space by inserting the filter *tcp and data*. One particular packet catches our attention, as it transmits on port 9999 a string PASS:string in clear text right before the data transmission on port 31337.

15.4 Solution

```
1 import io
2 import struct
3 import pyzipper
4 from scapy.all import rdpcap, TCP, Raw
5
6 #pass_dec = "49 63 79 46 6c 31 70 70 33 72 24 32 30 32 36"
7 password = b"IcyFl1pp3r$2026"
8
9 packets = rdpcap("challenge.pcap")
10 segments = []
11 for pkt in packets:
12     if pkt.haslayer(TCP) and pkt.haslayer(Raw) and pkt[TCP].dport == 31337:
13         segments.append((pkt[TCP].seq, bytes(pkt[Raw].load)))
14
15 segments.sort(key=lambda x:x[0])
16
17 done = set()
18 stream = b""
19 for seq, data in segments:
20     if seq not in done:
21         done.add(seq)
22         stream += data
23
24 magic = stream[:4]
25 xor_key = stream[4:4+16] #try 4,8,16,24,32
26 size = struct.unpack(">I", stream[20:20+4])
27 encrypted = stream[24:24+size[0]]
28 plaintext = bytes([a ^ xor_key[i%16] for i, a in enumerate(encrypted)])
29
```

```

30 #wrap bytes in a file-like object
31 zip_buffer = io.BytesIO(plaintext)
32
33 with pyzipper.AESZipFile(zip_buffer, 'r') as zip_ref:
34     try:
35         print(zip_ref.namelist())
36         zip_ref.setpassword(pwd=password)
37         print("Extraction successful!")
38         for file in zip_ref.namelist():
39             print(zip_ref.read(file).decode("utf-8"))
40     except RuntimeError as e:
41         print(f"Error: {e}")

```

Listing 41: exp.py